

Day 118 Hands-On Exercise — 8 Deliberately Different MLflow Runs

Use **one small classification dataset** and create **8 MLflow runs** where each run has a **clear reason for existing**.

The objective is not to chase the highest score. It is to practice:

change something



record the change



measure the result



compare with previous runs

MLflow's scikit-learn integration can log model parameters, metrics, fitted models, and other metadata; autologging also works with scikit-learn Pipeline objects. :chatgpt-content-reference{index="0"}

1. Dataset

For this exercise, imagine a small customer-purchase dataset:

Age	Income	City	Previous_Purchases	Purchased
22	28000	Delhi	1	No
35	52000	Mumbai	5	Yes
29	41000	Delhi	2	No
44	75000	Bengaluru	8	Yes
38	61000	Mumbai	6	Yes
26	35000	Bengaluru	1	No
51	90000	Delhi	10	Yes
31	47000	Mumbai	3	No

In a real exercise, use more rows than this. This table is only to understand the structure.

Features

Age

Income

City

Previous_Purchases

Target

Purchased

This is a:

binary classification problem

Yes / No

2. Basic preprocessing

You already learned this on Day 117 Part-4.

Your preprocessing could conceptually be:

Numerical columns

Age

Income

Previous_Purchases

↓

missing-value handling

↓

optional scaling

and:

Categorical column

City

↓

missing-value handling

↓

OneHotEncoding

Then:

preprocessing

↓

classifier

inside a scikit-learn Pipeline.

3. Create one MLflow experiment

Give the entire exercise one experiment name, for example:

customer_purchase_beginner_experiment

Inside it, create the eight runs below.

Think:

ONE EXPERIMENT

|— Run 1

|— Run 2

|— Run 3

- |— Run 4
- |— Run 5
- |— Run 6
- |— Run 7
- |— Run 8

Run 1 — Logistic Regression Baseline

Run name

01_logistic_baseline

Start with the simplest reasonable model.

Preprocessing

↓

Logistic Regression

Keep mostly default/basic settings.

Log as parameters

```
model_name = LogisticRegression
```

```
scaling = yes
```

```
class_weight = none
```

You may also log important Logistic Regression settings such as:

```
C
```

```
max_iter
```

Log as metrics

```
accuracy
```

```
precision
```

```
recall
```

```
F1
```

Purpose

This becomes your **baseline**.

Every later experiment can be compared with it.

Your thinking should be:

| Before changing anything, what performance do I get from a simple model?

Run 2 — Logistic Regression with Different c

Run name

02_logistic_C_change

Keep everything from Run 1 the same.

Change only:

C

For example:

Run 1:

C = 1.0

Run 2:

C = 0.1

You don't need to deeply understand regularization mathematics yet.

For now:

| C is a Logistic Regression hyperparameter that you can experimentally change.

Log

```
model_name = LogisticRegression
```

```
C = 0.1
```

```
scaling = yes
```

and:

accuracy

precision

recall

F1

Main question

Compare Run 1 and Run 2:

Did changing only C change the metrics?

This teaches:

same model

+

different hyperparameter

Run 3 — Logistic Regression with Class Weight

Run name

03_logistic_balanced

Go back to something close to your baseline but change:

class_weight

For example:

```
class_weight = "balanced"
```

Now compare:

Run 1

```
class_weight = None
```

vs

Run 3

```
class_weight = balanced
```

Pay particular attention to

precision

recall

F1

rather than only accuracy.

Why?

If one target class is less common, class weighting can change how the classifier treats the classes.

Main question

Did recall increase?

Did precision decrease?

What happened to F1?

What happened to accuracy?

Do not assume that every metric should improve together.

Run 4 — Decision Tree Baseline

Run name

04_tree_baseline

Now change the **model family**.

Use:

DecisionTreeClassifier

instead of Logistic Regression.

Keep preprocessing and train/test data as consistent as possible.

Log parameters

For example:

```
model_name = DecisionTreeClassifier  
max_depth = None  
class_weight = None
```

Metrics

Again use the same:

```
accuracy  
precision  
recall  
F1
```

Why use exactly the same metrics?

Because comparisons are easier when every run is evaluated consistently.

Main question

How does a Decision Tree behave compared with Logistic Regression?

You're not deciding permanently which model is "best."

You're observing differences.

Run 5 — Decision Tree with Limited Depth

Run name

05_tree_depth_3

Keep Run 4 almost identical but change:

```
max_depth = 3
```

Now:

Run 4

Decision Tree

```
max_depth = unlimited/default
```

Run 5

Decision Tree

```
max_depth = 3
```

Why?

A deep Decision Tree can learn very detailed patterns.

Limiting the depth makes the tree simpler.

Track

`max_depth = 3`

plus your normal metrics.

Bonus

Log:

`confusion_matrix.png`

as an artifact.

Now you're learning that an MLflow run can contain more than numerical metrics.

Run 6 — Decision Tree + Balanced Class Weight

Run name

`06_tree_depth3_balanced`

Start from Run 5:

`DecisionTree`

`max_depth = 3`

and add:

`class_weight = balanced`

Now compare:

Run 5:

`max_depth = 3`

`class_weight = None`

Run 6:

`max_depth = 3`

`class_weight = balanced`

You changed only one major thing.

That's good experimental practice.

Main questions

What happened to recall?

What happened to precision?

Did F1 change?

Did the confusion matrix change?

Run 7 — Random Forest Baseline

Run name

07_random_forest

Now introduce:

RandomForestClassifier

For a simple beginner run, perhaps use:

```
n_estimators = 100
```

Keep other settings reasonably simple.

Log parameters

```
model_name = RandomForestClassifier
```

```
n_estimators = 100
```

```
class_weight = None
```

Potentially:

```
max_depth
```

as well.

Log metrics

```
accuracy
```

```
precision
```

```
recall
```

```
F1
```

Artifact idea

If you generate one, log:

```
feature_importance.png
```

Don't worry about advanced interpretation yet.

Simply observe which features the model considered more useful.

Run 8 — Change Preprocessing

This run is especially useful because it teaches:

| ML experiments aren't only about changing models.

Run name

08_logistic_no_scaling

Return to Logistic Regression.

Use approximately the same configuration as Run 1, but change:

scaling = no

Compare:

Run 1

Missing-value handling

→ encoding

→ scaling

→ Logistic Regression

against:

Run 8

Missing-value handling

→ encoding

→ NO scaling

→ Logistic Regression

Log:

model_name = LogisticRegression

scaling = no

Then compare the metrics.

Main question

Did preprocessing affect the model?

This teaches an important lesson:

ML experiment

≠ only model experiment

It can also be:

preprocessing experiment

4. Your Final Eight Runs

Your experiment should eventually resemble this:

Run	Main change	Model	Important parameter
1	Baseline	Logistic Regression	C=1.0
2	Hyperparameter	Logistic Regression	C=0.1
3	Class weighting	Logistic Regression	class_weight=balanced
4	Different model	Decision Tree	default/basic

Run	Main change	Model	Important parameter
5	Tree hyperparameter	Decision Tree	max_depth=3
6	Class weighting	Decision Tree	depth 3 + balanced
7	Different model	Random Forest	n_estimators=100
8	Preprocessing	Logistic Regression	scaling off

This gives you practice with:

model comparison
 hyperparameters
 class weights
 preprocessing
 metrics
 artifacts
 run naming
 run comparison

5. Keep the Train/Test Split the Same

This is very important.

Suppose every run uses a different random split:

Run 1 → test customers A, B, C

Run 2 → test customers X, Y, Z

Then model comparison becomes harder because you changed both:

model/settings
 AND
 test data

For this exercise, use the same:

random_state

when creating the train/test split.

Conceptually:

One fixed train/test split

↓

Run 1
 Run 2
 Run 3
 ...
 Run 8

Now comparisons are cleaner.

6. What to Log in Every Run

Try to keep a consistent minimum set.

Parameters

```
model_name
preprocessing/scaling
important model hyperparameters
class_weight
```

Metrics

```
accuracy
precision
recall
F1
```

Tags

For example:

```
purpose = baseline
```

or:

```
purpose = hyperparameter_test
```

or:

```
purpose = preprocessing_test
```

Artifacts

For selected runs:

```
confusion_matrix.png
predictions.csv
feature_importance.png
```

Model

Save the fitted model/pipeline when appropriate.

MLflow's current scikit-learn autologging can automatically capture estimator parameters, common classifier metrics, and the fitted estimator; it can also track nested parameters inside meta-estimators such as pipelines. :chatgpt-content-reference{index="1"}

7. Manual Logging First

For your first few runs, think in this form:

```
start MLflow run
```

```
log:
```

model name

log:

important hyperparameters

train pipeline

predict test data

calculate:

accuracy

precision

recall

F1

log those metrics

save useful artifact

save/log model

end run

You don't need a lot of code.

The point is to understand:

What exactly belongs to this run?

8. Then Try Autologging

After you understand Runs 1–8 manually, repeat one model with:

```
mlflow.sklearn.autolog()
```

and inspect what appears automatically.

Current MLflow documentation says scikit-learn autologging records model parameters from `get_params()`, training/model metrics, and the fitted estimator; it supports scikit-learn pipelines as well. :chatgpt-content-reference{index="2"}

Your learning exercise becomes:

Manual logging

↓

"I understand what is being tracked."

Autologging

↓

"I can let MLflow capture much of it."

9. Beginner-Friendly Pseudocode

LOAD DATA

separate:

X

y

perform one fixed train/test split

CREATE MLFLOW EXPERIMENT

RUN 1:

create baseline preprocessing

create Logistic Regression

create Pipeline

train

predict

calculate metrics

log parameters

log metrics

RUN 2:

keep everything same

change Logistic C

train

evaluate

log

RUN 3:

keep Logistic Regression

use balanced class weight

train

evaluate

log

RUN 4:

change model to Decision Tree

train

evaluate

log

RUN 5:

```
keep Decision Tree
set max_depth = 3
train
evaluate
log confusion matrix
```

RUN 6:

```
keep depth = 3
add balanced class weight
train
evaluate
log
```

RUN 7:

```
use Random Forest
set n_estimators
train
evaluate
log
```

RUN 8:

```
return to Logistic Regression
remove scaling
train
evaluate
log
```

OPEN MLFLOW UI

COMPARE ALL RUNS

10. What Your Results Sheet Might Look Like

Do not copy these numbers—they are only an illustration.

Run	Model	Change	Accuracy	Precision	Recall	F1
1	Logistic	baseline	0.82	0.80	0.72	0.76
2	Logistic	C changed	0.81	0.78	0.74	0.76
3	Logistic	balanced	0.79	0.72	0.84	0.78
4	Tree	baseline	0.80	0.76	0.77	0.76
5	Tree	depth 3	0.83	0.80	0.78	0.79

Run	Model	Change	Accuracy	Precision	Recall	F1
6	Tree	balanced	0.81	0.75	0.86	0.80
7	Random Forest	100 trees	0.85	0.82	0.82	0.82
8	Logistic	no scaling	0.78	0.75	0.68	0.71

The exercise is **not**:

Find highest F1

→ finished

Instead, investigate:

Why did this metric change?

What exactly was different?

Did recall increase while precision decreased?

Did preprocessing matter?

Did limiting tree depth matter?

Did class weighting change minority-class behavior?

11. After All Eight Runs, Answer These Questions

Use the MLflow UI and your understanding to answer:

1. Which runs used the same model but different hyperparameters?
2. What happened when you changed C?
3. What happened when you changed class_weight?
4. How did Logistic Regression, Decision Tree, and Random Forest behave differently?
5. Did limiting Decision Tree depth change its test metrics?
6. Did removing scaling affect Logistic Regression?
7. Which run produced noticeably higher recall?
8. Which run produced noticeably higher precision?
9. Which run has a useful confusion-matrix artifact?
10. Can you identify exactly what was changed in every run without opening your training code?

That final question is especially important.

A well-tracked MLflow experiment should make:

"What did I do in this run?"

easy to answer.

12. Your Day 118 Mini-Project Structure

Keep it small:

mlflow_beginner_project/

```
|— data/
|   |— customers.csv
|
|— train.py
|
|— artifacts/
|   |— optional generated charts
|
|— README / notes
```

You do **not** need:

deployment
Docker
cloud hosting
Kubernetes
CI/CD
model serving
advanced tuning

for this exercise.

The learning path should simply be:

```
Build baseline
  ↓
Track it
  ↓
Change one thing
  ↓
Track again
  ↓
Repeat 8 times
  ↓
Open MLflow UI
  ↓
Compare parameters + metrics + artifacts
  ↓
Explain what each experiment taught you
```

That gives you a genuinely useful **first hands-on MLflow project** without making Day 118 unnecessarily advanced.